Meeting 6 — Execution Timing, Task Scheduling, Inter-Task Communication, and CPU Time Budget

Phase: 3.1 — Runtime Architecture Design
Meeting Objective: Define when every subsystem executes, how software modules communicate with one another, and how CPU execution time is allocated to guarantee uninterrupted real-time audio.

6.1 Purpose of This Meeting

The Digital Harmonium Workstation is fundamentally a **real-time embedded audio system**.

Unlike ordinary desktop software, where small execution delays are usually unnoticeable, an embedded musical instrument must continuously produce audio without interruption. Missing even a single audio deadline can introduce audible artifacts such as clicks, pops, crackles, or complete audio dropouts.

Therefore, before implementing the runtime, it is necessary to establish:

- How often each subsystem executes.
- Which subsystems have higher priority.
- How different software modules communicate.
- How CPU execution time is budgeted.
- Which operations are allowed inside the real-time path and which must remain in the background.

This meeting establishes those rules.

6.2 Fundamental Runtime Philosophy

The runtime is designed around one non-negotiable principle:

> **Audio generation always has the highest priority.**

Every other subsystem—including the user interface, display, SD card access, recording, diagnostics, and settings—exists to support the audio engine and must never delay it.

Accordingly, all software tasks are classified according to their required execution frequency.

6.3 Execution Frequency Classification

Every software component operates according to one of five timing classes.

Class A — Continuous Real-Time Tasks

These tasks are synchronized directly with audio generation.

Characteristics:

- Execute once for every audio block.
- Missing a deadline immediately produces audible glitches.
- Highest runtime priority.

Typical members:

- DMA Audio Engine
- Audio Mixer
- Voice Rendering
- Envelope Engine
- Filter Engine
- LFO Processing

Execution frequency:

- Every 512 audio samples
- Approximately every **11.6 ms** at 44.1 kHz

Class B — High-Speed Periodic Tasks

These tasks are not directly responsible for audio generation but must remain extremely responsive.

Characteristics:

- Execute every 1–2 milliseconds.
- Responsible for immediate user interaction.

Typical members:

- Keyboard Scanner
- Button Scanner
- LED Refresh

Execution frequency:

- 500–1000 Hz

Class C — Medium-Speed Periodic Tasks

These tasks update information intended for human observation.

Characteristics:

- Human vision cannot perceive updates faster than roughly 20–30 Hz.
- Therefore no benefit exists in refreshing faster.

Typical members:

- LCD Update
- Status Display
- Current Instrument Display
- Tempo Display

Execution frequency:

- Every 20–50 ms

Class D — Slow Background Tasks

These tasks perform maintenance.

Characteristics:

- Low urgency.
- Execute only occasionally.
- Never interfere with audio.

Typical members:

- Diagnostics
- Runtime Statistics
- Memory Monitoring
- CPU Usage Monitoring
- Internal Logging

Execution frequency:

- Every 100–1000 ms

Class E — Event-Driven Tasks

These tasks execute only when explicitly requested.

Characteristics:

- Consume no CPU while idle.
- Activated only by specific user or system events.

Typical members:

- Instrument Loading
- Recording Save
- USB Mode
- Settings
- SD Card Operations

Execution frequency:

- Event-driven only

6.4 Final Execution Frequency Table

Component	Execution Frequency
DMA Audio	Every 11.6 ms
Voice Rendering	Every 11.6 ms
Audio Mixer	Every 11.6 ms
Envelope Engine	Every 11.6 ms
Filter Engine	Every 11.6 ms
Keyboard Scanner	Every 1 ms
Button Scanner	Every 1–2 ms
LED Refresh	Every 2–5 ms
LCD Refresh	Every 20–50 ms
Recorder	Event + Buffer Driven
SD Card Writes	Background
Instrument Loader	Event

Component	Execution Frequency
USB Mode	Event
Diagnostics	Every 100–1000 ms

6.5 Inter-Module Communication Philosophy

Software modules should remain as independent as possible.

Direct dependencies between modules increase complexity and reduce maintainability.

Instead, modules communicate using dedicated communication mechanisms.

6.6 Communication Method 1 — Queues

Queues function as message mailboxes.

Instead of directly calling another subsystem, a task simply places a message into the queue.

Example:

Keyboard Scanner detects:

- Key Press
- Note Number
- Velocity

Rather than invoking the synthesizer directly, it pushes a **KEY_ON** event into the Audio Queue.

Later, the Audio Engine processes that message and allocates the required voice.

Advantages:

- Loose coupling
- Predictable execution
- Better modularity
- Safer scheduling

6.7 Communication Method 2 — Event Flags

Some information consists only of ON/OFF conditions.

Examples include:

- Recording Active
- USB Connected
- SD Mounted
- Tanpura Running
- Tabla Running

These conditions are represented as simple event flags.

Modules periodically check these flags rather than exchanging complex messages.

Advantages:

- Extremely lightweight
- Minimal CPU overhead
- Very fast synchronization

6.8 Communication Method 3 — Shared Runtime Variables

Certain system parameters change infrequently but are required by multiple modules.

Examples:

- Master Volume
- Current Instrument
- Global Tempo
- Global Transpose

Instead of repeatedly transmitting these values through queues, they are stored as shared runtime variables.

Any module requiring them simply reads the current value.

Advantages:

- Simple access
- Reduced communication overhead

- Centralized configuration

6.9 Communication Method 4 — Double Audio Buffering

Audio generation uses a double-buffer strategy.

Two buffers exist simultaneously.

While the DMA peripheral is transmitting one buffer to the audio output hardware, the CPU fills the second buffer with newly synthesized audio.

Once transmission completes, the two buffers swap roles.

This ensures that:

- CPU never overwrites data currently being transmitted.
- DMA never reads partially generated audio.
- Continuous playback is maintained.

This buffering strategy forms the foundation of stable real-time audio generation.

6.10 CPU Time Budget Philosophy

The ESP32 receives approximately **11.6 milliseconds** to generate each audio block.

Every subsystem must collectively complete its work within this deadline.

This available time is treated as the system's CPU budget.

If total execution exceeds the deadline, the DMA engine receives incomplete audio, producing audible artifacts.

Therefore every subsystem must have an allocated execution budget.

6.11 Recommended CPU Budget Allocation

Runtime Component	Approximate Budget
Voice Rendering	5–6 ms
Audio Mixer	1–2 ms
Audio Effects	~1 ms
Keyboard Scanner	<0.5 ms
Button / LED Processing	<0.5 ms
Queue Processing	<0.5 ms
Safety Margin	~2 ms

6.12 Importance of Safety Margin

Real systems experience occasional bursts of workload.

For example:

- Multiple simultaneous key presses.
- Tabla advances to the next beat.
- Tanpura updates.
- Recording writes buffered events.
- LCD refresh occurs.

If the runtime operates continuously at 100% CPU utilization, these temporary spikes immediately exceed the available execution time.

Therefore a permanent reserve of approximately **2 milliseconds** is intentionally left unused under normal conditions.

This safety margin allows the workstation to absorb transient workload increases without affecting audio quality.

6.13 Real-Time Scheduling Rules

The runtime shall obey the following scheduling principles:

1. Audio generation always receives highest execution priority.

2. UI updates must never delay audio rendering.
3. Background tasks execute only after higher-priority work completes.
4. Long-running operations (instrument loading, SD card access, recording save, diagnostics) are prohibited from executing inside the real-time audio path.
5. Every software component must have a clearly defined execution frequency and execution budget.

6.14 Meeting Outcome

This meeting establishes the timing philosophy governing the entire Digital Harmonium Workstation runtime.

The workstation will therefore operate using:

- Five execution frequency classes.
- Four standardized communication mechanisms.
- Double-buffered audio generation.
- Explicit CPU time budgeting.
- Strict real-time scheduling rules.
- Guaranteed priority for uninterrupted audio playback.

These principles will serve as the foundation for all subsequent runtime task implementation and FreeRTOS scheduling decisions during Phase 3.